



마음나래_2단계 기획 및 설계

설계의 4가지 단계

1. 기능 목록 만들기

프로그램이 해야 할 일을 "동사 + 목적어" 형태로 모두 적습니다. 그리고 우선순위를 매기세요: 반드시(MUST), 있으면 좋음(NICE), 시간 남으면(LATER)

- 감정, 정신상태, 현재 행동 상태 등 극복하고 싶은 자신의 상태를 입력받기(MUST)
- 사용자의 입력이 모호하거나 더 구체적인 상태를 질문하기(MUST)
- 입력받은 데이터를 통해 인지/정서/행동의 측면에서 해결방안을 단기 측면에서 추천하기(MUST)
- 단기 측면의 해결방안 추천 이후 장기 측면의 해결방안을 원하는지 묻기(NICE)
- 장기적인 측면의 해결방안을 원할 경우, 인지/정서/행동의 측면에서 각각 하나씩 제시하기(NICE)
- 사용자가 선택한 하나의 장기적 해결방안을 5단계로 나누어 실천 방안 제시하기(LATER)
- 사용자의 대화창 기록할 때 "마음상태 + 선택한 해결방안이 어떤 종류인지"를 기준으로 표시하기(LATER)
- 사용자가 자신의 변화 기록하여 성장을 눈으로 볼 수 있도록, 해결방안 사용 이후 변화점을 기록하는 질문하기(LATER)

2. 데이터 설계- 어떤 모양으로 저장

프로그램의 정보를 어떤 자료구조에 담을지 정하세요. 비전공자에게 가장 쉬운 출발점은 리스트와 딕셔너리입니다. (어떤 변수에 무엇이 들어가는지)

변수명 (Variable Name)	데이터 타입 (data type)	변수에 들어갈 내용 (description)	예시(Example)
user_input	문자열(String)	사용자가 입력한 정서/신체 증상 문장 전체	"나 너무 무기력해. 아무것도 안하고 싶어"
system_instruction	문자열(String)	AI에게 심리상담사 역할을 부여하는 지시문(프롬프트)	너는 심리상담사 마음나래야. [인지], [정서], [행동]으로 나눠서 사용자의 질문에 맞는 해결방안을 알려줘
conversation_history	리스트(List)	사용자의 입력과 AI의 답변을 순서대로 누적해 기록하는 대화 저장소	[{"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]
ai_raw_response	딕셔너리(Dict)	AI API가 서버로부터 보내온 날것의 응답 데이터(복잡한 구조)	{"id": "chatcmpl-...", "choices": [...], ...}
Final_solution	문자열(String)	AI응답 중 핵심 답변([인지/ 정서/행동])만 골라낸 최종 문장	"[인지] ... \n[정서] ... \n[행동] ..."

***API: Application Programming Interface:**

프로그램끼리 대화할 수 있게 해주는 규칙과 창구

3. 함수 설계- 누가 무엇을 할까?

각 기능을 하나의 함수로 잡고, 입력(매개변수)과 출력(반환값)을 미리 정하세요. 이 단계에서 "내부 구현"은 비워둬도 됩니다 — 이걸 의사코드(pseudocode)라고 합니다.

```
def initialize_program():
    print("마음나래입니다. 오늘은 무슨 마음이 들었나요?")
    # 초기 리스트 생성
    history = [{"role": "system", "content": "너는 심리상담사야..."}]
    return history

def get_user_input():
    return input("현재 마음 상태를 알려주세요: ")

def request_ai_solution(history):
    # 실제 개발 시에는 여기에 OpenAI API 호출 코드가 들어갑니다.
    response = "AI가 생성한 [인지][정서][행동] 지침 문장"
    return response

def print_solution(solution):
    print("\n[마음나래의 처방전]")
    print(solution)
    print("-" * 40)
```

메인 시스템 흐름

```
def initialize_program():
    print("마음나래입니다. 오늘은 무슨 마음이 들었나요?")
    # 초기 리스트 생성
    history = [{"role": "system", "content": "너는 심리상담사야..."}]
    return history

def get_user_input():
    return input("현재 마음 상태를 알려주세요: ")

def request_ai_solution(history):
    # 실제 개발 시에는 여기에 OpenAI API 호출 코드가 들어갑니다.
    response = "AI가 생성한 [인지][정서][행동] 지침 문장"
    return response

def print_solution(solution):
    print("\n[마음나래의 처방전]")
    print(solution)
    print("-" * 40)

#메인 시스템 흐름

def main():
    # 1. 초기화 (리스트 생성)
```

```

conversation_history = initialize_program()

while True:
    # 2. 입력 받기
    user_text = get_user_input()
    if user_text == "종료":
        break

    # 3. 대화 기록(리스트)에 사용자 입력(딕셔너리) 추가
    conversation_history.append({"role": "user", "content": user_text})

    # 4. AI에게 대화 기록을 넘겨주고 답변 받기
    ai_answer = request_ai_solution(conversation_history)

    # 5. 답변 출력
    print_solution(ai_answer)

    # 6. 대화 기록(리스트)에 AI 답변(딕셔너리)도 추가하여 기억하게 함
    conversation_history.append({"role": "assistant", "content": ai_answer})

```

4. 화면 흐름 설계 -사용자는 무엇을 보게 될까?

사용자가 프로그램을 실행했을 때 무엇이 보이고, 무엇을 입력하고, 다음에 무엇이 나오는지 종이에 그려보세요. 손그림이면 충분합니다.



